

BigQuery Data Warehouse Notes

PostgreSQL → BigQuery Conversion — Reference & File Guide

This document describes every file inside the **bigquery/** folder: what it does, how it maps back to the original PostgreSQL file in **pgsql/**, and how the pieces fit together into one end-to-end pipeline — GCS landing → bronze → silver → gold → Cloud Composer scheduling — plus a full PL/pgSQL → BigQuery Scripting concept-by-concept reference.

Folder	Contents / purpose
datasets/	Source CSVs (source_crm, source_erp) to upload to GCS.
gcs_load/	GCS landing-zone docs + LOAD DATA / bq load commands (new step; Postgres had none).
bronze/	Raw-layer DDL + load_bronze() procedure (GCS to bronze).
silver/	Cleansed-layer DDL + load_silver() procedure (bronze to silver).
gold/	Star-schema DDL + load_gold() procedure (silver to gold).
procedure_bigquery/	9 topic-by-topic BigQuery Scripting tutorials + a full mini ETL project.
BIGQUERY PROJECT/	The complete worked project: DDL, procedures, GCS commands, 5 Composer DAGs.

1. Pipeline Architecture

The whole warehouse follows the classic **medallion architecture** (bronze → silver → gold), scheduled end-to-end by Cloud Composer (managed Apache Airflow). The one step that has no PostgreSQL equivalent is landing the CSVs in Google Cloud Storage first, since BigQuery -- unlike Postgres' `COPY . . . FROM 'local path'` -- can only load data from GCS (or Drive, or inline values), never from a worker's local disk.

- 1 **datasets/source_crm, datasets/source_erp** (local CSVs, or the flat CSVs in **BIGQUERY PROJECT**) are uploaded to a GCS bucket — see `gcs_load/README.md`.
- 2 **gs://your-bucket/datasets/...** is the GCS landing zone.
- 3 **LOAD DATA / bq load / the GCSToBigQueryOperator** Airflow task loads the CSVs into the **bronze** dataset (raw, largely untyped landing tables).
- 4 **CALL bronze.load_bronze()** (or **bronze.dwh_load_crm_erp_bronze()** in the **BIGQUERY PROJECT** variant) truncates/prepares the bronze tables.
- 5 **CALL silver.load_silver()** (or **silver.dwh_data_load_silver()**) cleans, standardizes, and deduplicates bronze rows into the **silver** dataset.
- 6 **CALL gold.load_gold()** (or **gold.dwh_load_todays_data()**) builds the star schema (**dim_customers, dim_products, fact_sales**) in the **gold** dataset.
- 7 A BI tool (Looker Studio, Looker, etc.) reads from **gold**.

Every arrow above is one Cloud Composer task. See **BIGQUERY PROJECT/Script/dag_gcs_to_bronze.py, dag_bronze_prepare.py, dag_silver_load.py, and dag_gold_load.py** — each DAG calls a BigQuery procedure through **BigQueryInsertJobOperator** (or loads files through **GCSToBigQueryOperator**), the direct replacement for the original PostgreSQL DAGs' **PostgresOperator** calls.

One-time setup (placeholders `your_gcp_project / your -bucket` must be replaced everywhere before running):

```
bq mk --dataset --location=US your_gcp_project:bronze
bq mk --dataset --location=US your_gcp_project:silver
bq mk --dataset --location=US your_gcp_project:gold
bq mk --dataset --location=US your_gcp_project:audit # used by procedure_bigquery/project/
gsutil -m cp -r datasets/source_crm gs://your-bucket/datasets/
gsutil -m cp -r datasets/source_erp gs://your-bucket/datasets/
```

2. File-by-File Notes

Every file below mirrors a same-named (or clearly corresponding) file in **pgsql/**. "PG source" gives that original file's path for side-by-side comparison.

2.1 datasets/ — source CSVs (PG source: **pgsql/datasets/**)

File	Description
source_crm/cust_info.csv	Raw CRM customer records (id, key, name, marital status, gender, create date) — input to bronze.crm_cust_info.
source_crm/prd_info.csv	Raw CRM product records (id, key, name, cost, line, start/end dates) — input to bronze.crm_prd_info.
source_crm/sales_details.csv	Raw CRM sales transactions (order/ship/due dates as integers, sales, qty, price) — input to bronze.crm_sales_details.
source_erp/CUST_AZ12.csv	Raw ERP customer demographics (id, birthdate, gender) — input to bronze.erp_cust_az12.
source_erp/LOC_A101.csv	Raw ERP customer location/country codes — input to bronze.erp_loc_a101.
source_erp/PX_CAT_G1V2.csv	Raw ERP product category/subcategory/maintenance flags — input to bronze.erp_px_cat_g1v2.

2.2 gcs_load/ — GCS landing step (new; no PG equivalent)

File	Description
README.md	Explains the GCS bucket/folder layout that mirrors datasets/, and the two ways to load GCS into bronze (LOAD DATA statement vs. bq load CLI vs. Airflow's GCSToBigQueryOperator).
load_data_gcs_to_bronze.sql	One LOAD DATA OVERWRITE ... FROM FILES(...) statement per bronze table — the direct BigQuery-DDL equivalent of PostgreSQL's COPY ... FROM 'local path'.
bq_load_cli_commands.sh	Shell script using the bq load CLI (plus gsutil cp to upload) to load every source CSV into its bronze table — equivalent of the psql \copy workflow in PGSQL PROJECT/Script/copy cmd.txt.

2.3 bronze/ — raw landing layer (PG source: **pgsql/bronze/**)

File	Description
ddl_bronze.sql	CREATE TABLE statements for the 6 bronze tables (crm_cust_info, crm_prd_info, crm_sales_details, erp_loc_a101, erp_cust_az12, erp_px_cat_g1v2), with a PostgreSQL-to-BigQuery type-mapping note (INT→INT64, VARCHAR→STRING, etc.).
proc_load_bronze.sql	CREATE PROCEDURE bronze.load_bronze() — truncates each bronze table then loads it via LOAD DATA OVERWRITE FROM FILES(gs://...), timing each step and logging progress via SELECT ... AS log_message (replacing RAISE NOTICE).

2.4 silver/ — cleansed layer (PG source: **pgsql/silver/**)

File	Description
ddl_silver.sql	CREATE TABLE statements for the 6 silver tables, each with a dwh_create_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP() column (BigQuery column DEFAULT works the same way as Postgres).

File	Description
<code>proc_load_silver.sql</code>	CREATE PROCEDURE <code>silver.load_silver()</code> — cleans and standardizes every bronze table into silver: trims/cases names, maps marital-status and gender codes, derives product category id and end-date via <code>LEAD()</code> , parses <code>YYYYMMDD</code> integer dates with <code>PARSE_DATE</code> , and recomputes sales/price using <code>SAFE_DIVIDE</code> .

2.5 gold/ — star schema layer (PG source: `pgsql/gold/`)

File	Description
<code>ddl_gold.sql</code>	CREATE TABLE statements for <code>gold.dim_customers</code> , <code>gold.dim_products</code> , and <code>gold.fact_sales</code> (the reporting-ready star schema).
<code>proc_load_gold.sql</code>	CREATE PROCEDURE <code>gold.load_gold()</code> — truncates and rebuilds all three gold tables from silver via LEFT JOINS, generating surrogate keys with <code>ROW_NUMBER()</code> and filtering <code>fact_sales</code> to the current month with <code>DATE_TRUNC(col, MONTH)</code> .

2.6 procedure_bigquery/ — BigQuery Scripting tutorials (PG source: pgsql/procedure_pgsql/)

File	Description
01_variables_and_datatypes.sql	DECLARE/SET syntax, type mapping (INT64/STRING/NUMERIC/BOOL/DATE), the 'no CONSTANT keyword' and 'no %TYPE' gaps, and STRUCT as the RECORD replacement.
02_if_else_control.sql	IF/ELSEIF/ELSE and the searched CASE statement (note: ELSIF becomes ELSEIF, one word).
03_for_loop.sql	FOR ... IN (query) DO ... END FOR over query results; recreates integer ranges with UNNEST(GENERATE_ARRAY(...)) since BigQuery has no native 'FOR i IN 1..5'; LEAVE/ITERATE replace EXIT/CONTINUE, with labelled-loop support.
04_while_loop.sql	WHILE ... DO ... END WHILE and the bare infinite LOOP ... END LOOP (replacing WHILE TRUE); shows the IF+LEAVE pattern replacing EXIT WHEN.
05_exception_handling_basic.sql	BEGIN ... EXCEPTION WHEN ERROR THEN ... END; explains BigQuery's single catch-all handler (no named exception classes like division_by_zero) and the @@error.message / @@error.statement_text system variables replacing SQLERRM/SQLSTATE.
06_cursors.sql	Reference file explaining BigQuery has NO cursor object at all (no REFCURSOR, OPEN/FETCH/CLOSE, bound cursors) and why (columnar OLAP engine); shows the FOR-loop-over-a-query pattern that replaces simple cursor usage.
07_exception_handling_advanced.sql	Nested BEGIN blocks for error isolation, RAISE USING MESSAGE = ... (BigQuery's only RAISE form; no ERRCODE/DETAIL/HINT), and reading @@error.* directly instead of GET STACKED DIAGNOSTICS.
08_functions.sql	Scalar SQL UDF as a single CASE expression (with ERROR() replacing RAISE EXCEPTION for validation); OUT/INOUT parameters moved from a function into a PROCEDURE (BigQuery functions can't have them); CREATE TABLE FUNCTION replacing RETURNS TABLE.
09_triggers.sql	Reference + runnable example explaining BigQuery has NO trigger object at all; shows the three replacement patterns (inline stamping, explicit audit-log INSERT in the same procedure, and Cloud Function/Eventarc for true event-driven automation) using a students_table + student_logs worked example.

2.7 procedure_bigquery/project/ — mini end-to-end ETL exercise (PG source: pgsql/procedure_pgsql/project/)

File	Description
01_ddl.sql	Creates the bronze.customer/products/sales, silver.dim_customer/dim_products/fact_products_sales, and audit.load_logs tables used by the exercise; notes the ROW_NUMBER()/GENERATE_UUID() replacements for SERIAL and FOREIGN KEY ... NOT ENFORCED for referential metadata.
02_dml.sql	Inserts deliberately messy mock rows (extra whitespace, mixed date formats, currency-symbol prices, an invalid customer row) into the bronze tables to exercise the cleaning logic.
03_load_silver_procedure.sql	The single-file 'everything in one script' teaching version: all 3 helper functions + the trigger-replacement note + the full silver.load_silver_data() procedure, covering all 8 procedural concepts in one place (mirrors the PG tutorial's monolithic layout).
04_run_project.sql	Runs CALL silver.load_silver_data() and then queries audit.load_logs, silver.dim_customer, silver.dim_products, and silver.fact_products_sales to inspect the results, with commentary on what each cleaning step changed.

File	Description
func_clean_gender.sql	CREATE FUNCTION silver.clean_gender(String) RETURNS STRING — one CASE expression standardizing 'm'/'male'/'f'/'female' variants to 'Male'/'Female'/'n/a'.
func_clean_price.sql	CREATE FUNCTION silver.clean_price(String) RETURNS NUMERIC — strips '\$', handles 'free'/blank/NULL, and uses SAFE_CAST + COALESCE (replacing a nested BEGIN/EXCEPTION block) to fall back to 0.00 on any bad input.
func_parse_date_safe.sql	CREATE FUNCTION silver.parse_date_safe(String) RETURNS DATE — tries YYYY-MM-DD, DD-MM-YYYY, then YYYY/MM/DD via COALESCE(SAFE.PARSE_DATE(...), ...), replacing three nested BEGIN/EXCEPTION fallback blocks.
proc_load_dim_customer.sql	CREATE PROCEDURE silver.load_dim_customer(IN proc_name, OUT inserted_rows) — full reload of silver.dim_customer from bronze.customer using SPLIT()/REGEXP_CONTAINS/the two functions above, with @@row_count and an EXCEPTION-WHEN-ERROR audit-log write.
proc_load_dim_products.sql	CREATE PROCEDURE silver.load_dim_products(IN proc_name, OUT inserted_rows) — full reload of silver.dim_products from bronze.products using clean_price(), REGEXP_CONTAINS validation, and the same audit-logging pattern.
proc_load_fact_sales.sql	*** KEY CONVERSION FILE *** CREATE PROCEDURE silver.load_fact_sales(...) — rewrites the PG cursor-based row-by-row loader as a set-based JOIN staged into a TEMP TABLE, a single INSERT ... SELECT ... WHERE for accepted rows, and a FOR loop over only the rejected rows to reproduce the per-row RAISE WARNING skip messages.
proc_load_silver_data.sql	CREATE PROCEDURE silver.load_silver_data() — master orchestrator that CALLs the three load procedures above plus validate_load() and print_load_summary(), isolating each step's errors in its own BEGIN/EXCEPTION block.
proc_print_load_summary.sql	CREATE PROCEDURE silver.print_load_summary(IN run_start_time) — FOR loop over audit.load_logs since run_start_time, formatting a pass/fail summary line per step with RPAD.
proc_validate_load.sql	CREATE PROCEDURE silver.validate_load() — WHILE loop over a 3-table checklist, using EXECUTE IMMEDIATE 'SELECT COUNT(*) FROM ' table_name INTO count for dynamic SQL row counts.
trigger_update_timestamp.sql	Reference-only note (no CREATE TRIGGER exists in BigQuery): explains that DEFAULT CURRENT_TIMESTAMP() already covers insert-time stamping for this project's always-full-reload tables, and shows the inline UPDATE/MERGE stamping pattern to use if an incremental path is ever added.

2.8 BIGQUERY PROJECT/ root — flat source CSVs (PG source: pgsql/PGSQL PROJECT/*.csv)

File	Description
CUST_AZ12.csv, LOC_A101.csv, PX_CAT_G1V2.csv	Copies of the ERP source CSVs, laid out flat (not in source_crm/source_erp subfolders) to match the original PGSQL PROJECT layout and the paths used by Scrpit/gcs_load_cmds.sh.
cust_info.csv, prd_info.csv, sales_details.csv	Copies of the CRM source CSVs, same flat layout, feeding Scrpit/gcs_load_cmds.sh and the bronze tables in Scrpit/DDL.sql.

2.9 BIGQUERY PROJECT/Scrpit/ — the full worked project (PG source: pgsql/PGSQL PROJECT/Scrpit/)

File	Description
DDL.sql	Full warehouse DDL in one file: all 6 bronze tables, all 6 silver tables, the 3 gold tables built with CREATE TABLE AS SELECT, and bronze.load_audit_log. Converted from DDL.txt; notes that CREATE DATABASE/SCHEMA have no BigQuery equivalent (project = database, dataset created via bq mk).
bronze.sql	CREATE PROCEDURE bronze.dwh_load_crm_erp_bronze() — truncates all 6 bronze tables only, leaving the actual row-loading to a separate GCSToBigQueryOperator task (same split of responsibility as the original bronze.txt + Airflow MinIO task).
silver_proc.sql	CREATE PROCEDURE silver.dwh_data_load_silver() — the alternate full bronze-to-silver cleanse (INITCAP names, SAFE_DIVIDE price fallback, DATE_SUB for product end-dates). Converted from silver_proc.txt.
gold.sql	CREATE PROCEDURE gold.dwh_load_todays_data() — incremental 'load only today's new rows' pattern using MAX(surrogate_key)+ROW_NUMBER() and DATE(dwh_create_date) = CURRENT_DATE() filters. Converted from gold.txt.
gcs_load_cmds.sh	bq load CLI commands (with a gsutil cp upload step first) loading each flat CSV in this folder into its bronze table — the direct replacement for the psql \copy session in copy cmd.txt.
dag_bronze_prepare.py	Composer DAG calling bronze.dwh_load_crm_erp_bronze() via BigQueryInsertJobOperator on a daily 23:00 schedule. Converted from the PostgresOperator version of the same name.
dag_silver_load.py	Composer DAG calling silver.dwh_data_load_silver() via BigQueryInsertJobOperator, same schedule.
dag_gold_load.py	Composer DAG calling gold.dwh_load_todays_data() via BigQueryInsertJobOperator, same schedule.
dag_gcs_to_bronze.py	*** KEY CONVERSION FILE *** Replaces dag_minio_to_bronze.py's hand-written MinIO+psycopg2 PythonOperator with one declarative GCSToBigQueryOperator task per source file, reading from a GCS bucket instead of a self-hosted MinIO server.

3. PostgreSQL → BigQuery Cheat Sheet

The condensed version of the full table in `bigquery/README.md` — every substitution pattern used across this folder, in one place.

PostgreSQL	BigQuery
<code>CREATE PROCEDURE ... LANGUAGE plpgsql AS \$\$... \$\$;</code>	<code>CREATE PROCEDURE ... BEGIN ... END;</code>
<code>schema</code>	<code>dataset</code> (created with <code>bq mk --dataset</code> , never in SQL)
<code>INT / VARCHAR(n) / TEXT / BOOLEAN</code>	<code>INT64 / STRING / STRING / BOOL</code>
<code>SERIAL / auto-increment</code>	No equivalent — <code>ROW_NUMBER()</code> for sequential keys, <code>GENERATE_UUID()</code> for opaque IDs
<code>x := val; / DECLARE x TYPE := val;</code>	<code>SET x = val; / DECLARE x TYPE DEFAULT val;</code>
<code>RAISE NOTICE '...', x;</code>	<code>SELECT FORMAT('...', x) AS log_message;</code> (or write to an audit table)
<code>ELSIF</code>	<code>ELSEIF</code>
<code>FOR i IN 1..5 LOOP</code>	<code>FOR r IN (SELECT i FROM UNNEST(GENERATE_ARRAY(1,5)) AS i) DO</code>
<code>EXIT / CONTINUE / EXIT WHEN cond;</code>	<code>LEAVE / ITERATE / IF cond THEN LEAVE; END IF;</code>
<code>EXCEPTION WHEN division_by_zero ...</code>	<code>EXCEPTION WHEN ERROR THEN</code> (one catch-all; branch on <code>@@error.message</code>)
<code>SQLERRM / SQLSTATE</code>	<code>@@error.message / @@error.formatted_stack_trace</code>
<code>RAISE EXCEPTION 'msg' USING ERRCODE=..., DETAIL=...;</code>	<code>RAISE USING MESSAGE = 'msg';</code> (fold detail into the text)
<code>Cursors (DECLARE CURSOR, OPEN/FETCH/CLOSE)</code>	Not supported — rewrite as a set-based JOIN (+ TEMP TABLE staging)
<code>Function body = full procedural block</code>	SQL UDF body = one expression; use <code>CASE + ERROR()</code> for logic/validation
<code>Function OUT/INOUT params</code>	Not supported on functions — use a PROCEDURE instead
<code>RETURNS TABLE (...) ... RETURN QUERY SELECT</code>	<code>CREATE TABLE FUNCTION ... AS (SELECT ...);</code>
<code>Triggers (CREATE TRIGGER BEFORE/AFTER ...)</code>	Not supported — stamp/log inline in the same UPDATE/MERGE/procedure
<code>COPY table FROM 'local/path.csv' WITH (...)</code>	<code>LOAD DATA OVERWRITE ds.table FROM FILES(format='CSV', uris=['gs://...'])</code>
<code>psql \copy ...</code>	<code>bq load --source_format=CSV ...</code>
<code>TO_DATE(x, 'YYYYMMDD')</code>	<code>PARSE_DATE('%Y%m%d', x)</code>
<code>SUBSTRING(x FROM a FOR b)</code>	<code>SUBSTR(x, a, b)</code>
<code>SPLIT_PART(x, ' ', n)</code>	<code>SPLIT(x, ' ')[SAFE_OFFSET(n-1)]</code>
<code>x ~ '^[0-9]+\$'</code>	<code>REGEXP_CONTAINS(x, r'^[0-9]+\$')</code>
<code>date_expr - INTERVAL '1 day'</code>	<code>DATE_SUB(date_expr, INTERVAL 1 DAY)</code>
<code>DATE_TRUNC('month', col)</code>	<code>DATE_TRUNC(col, MONTH)</code>

PostgreSQL	BigQuery
clock_timestamp()	CURRENT_TIMESTAMP()
EXTRACT(EPOCH FROM (t2 - t1))	TIMESTAMP_DIFF(t2, t1, SECOND)
GET DIAGNOSTICS x = ROW_COUNT;	SET x = @@row_count;
EXECUTE 'sql' INTO var;	EXECUTE IMMEDIATE 'sql' INTO var;
DELETE FROM t; (no WHERE required)	DELETE FROM t WHERE TRUE; (WHERE clause required)

4. What Was Intentionally Skipped

PG source file	Why it was skipped
procedure_pgsql/logs/dbt.log	Empty tool-generated log artifact from a local dbt run — not course content, nothing to convert.
procedure_pgsql/logs/query_log.sql	Empty file — not course content, nothing to convert.
PGSQL PROJECT/Scrpt/airflow_dags_11pm_pipeline.zip	A zip bundle of the 4 DAG files. Superseded by the plain dag_*.py files in BIGQUERY PROJECT/Scrpt/ — Cloud Composer syncs DAGs from a dags/ GCS folder directly, it does not consume zip bundles.

Before running anything: every file with `gs://your-bucket` or `your_gcp_project` needs those placeholders replaced with your real GCP project ID and bucket name.